

## Chapter 5

# Designing Our Trading Ecosystem

---

*Building Low-Latency Applications with C++*

Zhijie Gao - Qishi C++ Study Group

```
OrderBook book_;

LFQueue<MReq> q_;

void run() {

    while(true)

        processNext();

}
```

# AGENDA

01

## Ecosystem Topology

Exchange vs. market participant components

02

## Matching Engine Design

Order book, FIFO matching, threading model

03

## Market Data Publisher

Incremental & snapshot streams, UDP multicast

04

## Order Gateway Server

TCP connections, FIFO sequencer, protocol codec

05

## Market Participant Side

Market data consumer & order gateway client

06

## Trading Strategy Framework

Order book, feature engine, risk manager

# ECOSYSTEM TOPOLOGY

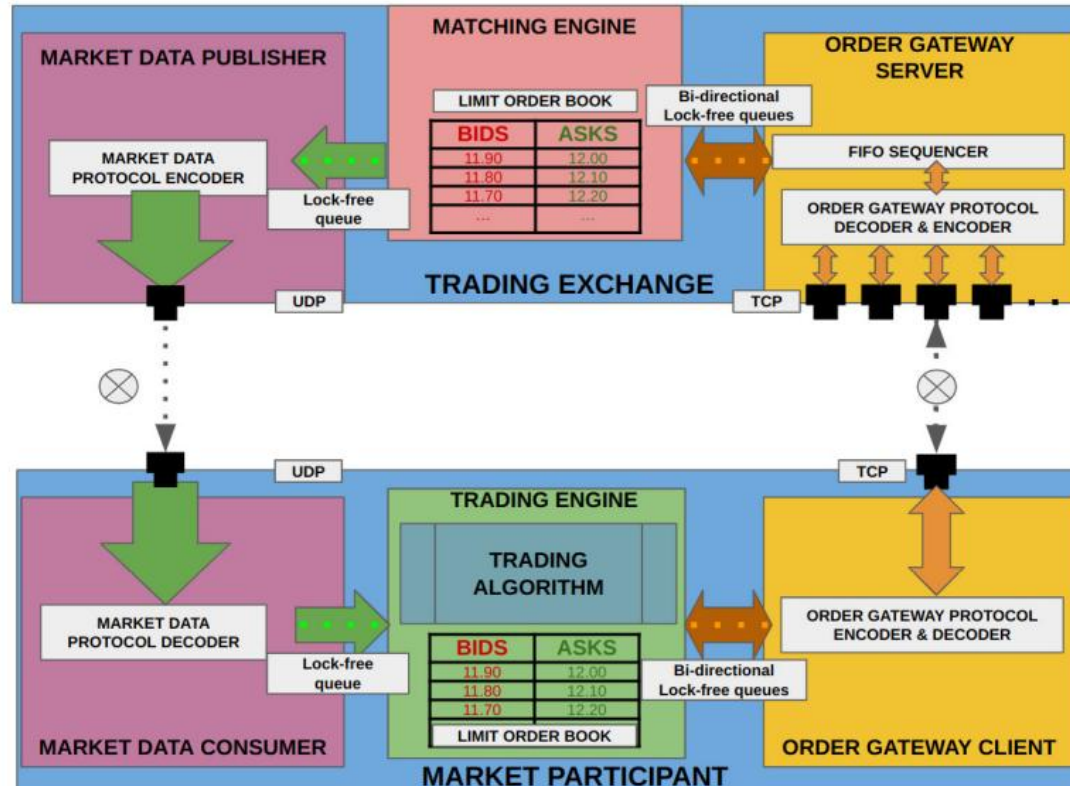


Figure 5.1 – Topology of a simple electronic trading ecosystem

# MATCHING ENGINE

The heart of the trading exchange



# ORDER BOOK MECHANICS — INITIAL STATE

Step 1 of 5

| BIDS (Buy Orders)    |                        |
|----------------------|------------------------|
| Client A – OrderId 1 | <b>BUY 20 @ 10.90</b>  |
| Client A – OrderId 2 | <b>BUY 10 @ 10.80</b>  |
| Client B – OrderId 3 | <b>BUY 5 @ 10.80</b>   |
| Client C – OrderId 4 | <b>BUY 100 @ 10.70</b> |

| ASKS (Sell Orders)   |                        |
|----------------------|------------------------|
| Client B – OrderId 5 | <b>SELL 10 @ 11.00</b> |
| Client C – OrderId 6 | <b>SELL 5 @ 11.00</b>  |
| Client B – OrderId 7 | <b>SELL 5 @ 11.10</b>  |
|                      |                        |

**Key rules shown here:**

- Bids sorted highest→lowest price
- Asks sorted lowest→highest price
- Same-price orders ordered by FIFO arrival time
- Spread = lowest ask (11.00) – highest bid (10.90) = 0.10

# RULE: ADDING ORDERS — FIFO PRIORITY

Step 2 of 5

Actions: Client A adds BUY 10 @ 10.90 | Client B adds SELL 10 @ 11.20

| BIDS (Buy Orders)          |     |             |
|----------------------------|-----|-------------|
| Client A – OrderId 1       | BUY | 20 @ 10.90  |
| Client A – OrderId 8 ★ NEW | BUY | 10 @ 10.90  |
| Client A – OrderId 2       | BUY | 10 @ 10.80  |
| Client B – OrderId 3       | BUY | 5 @ 10.80   |
| Client C – OrderId 4       | BUY | 100 @ 10.70 |

| ASKS (Sell Orders)         |      |            |
|----------------------------|------|------------|
| Client B – OrderId 5       | SELL | 10 @ 11.00 |
| Client C – OrderId 6       | SELL | 5 @ 11.00  |
| Client B – OrderId 7       | SELL | 5 @ 11.10  |
| Client B – OrderId 9 ★ NEW | SELL | 10 @ 11.20 |



## FIFO Rule

OrderId 8 joins behind OrderId 1 at the same price of 10.90 because it arrived later. First In = First priority at each price level.



New order added

# RULE: MODIFYING ORDERS — PRIORITY IMPACT

Step 3 of 5

Actions: Client A increases OrderId 2 qty 10→20 (loses priority) | Client B reduces OrderId 5 qty 10→1 (keeps priority)

| BIDS (Buy Orders)           |                 |
|-----------------------------|-----------------|
| Client A – OrderId 1        | BUY 20 @ 10.90  |
| Client A – OrderId 8        | BUY 10 @ 10.90  |
| Client B – OrderId 3        | BUY 5 @ 10.80   |
| Client A – OrderId 2 ↓ qty+ | BUY 20 @ 10.80  |
| Client C – OrderId 4        | BUY 100 @ 10.70 |

## ⚠ Increase Quantity

OrderId 2: qty increased 10 → 20. Order is moved to the back of the 10.80 bid queue — priority is lost (treated like a new order).

| ASKS (Sell Orders)          |                 |
|-----------------------------|-----------------|
| Client B – OrderId 5 ↓ qty- | SELL 1 @ 11.00  |
| Client C – OrderId 6        | SELL 5 @ 11.00  |
| Client B – OrderId 7        | SELL 5 @ 11.10  |
| Client B – OrderId 9        | SELL 10 @ 11.20 |

## ✅ Reduce Quantity

OrderId 5: qty reduced 10 → 1. Order stays at the front of the 11.00 ask queue — priority is preserved.

Modified order

# RULE: PRICE CHANGE = CANCEL + REINSERT

Step 4 of 5

Actions: Client C moves OrderId 4 price 10.70→10.90 | Client B cancels OrderId 9 (SELL 10 @ 11.20)

| BIDS (Buy Orders)            |     |             | ASKS (Sell Orders)               |      |            |
|------------------------------|-----|-------------|----------------------------------|------|------------|
| Client A – OrderId 1         | BUY | 20 @ 10.90  | Client B – OrderId 5             | SELL | 1 @ 11.00  |
| Client A – OrderId 8         | BUY | 10 @ 10.90  | Client C – OrderId 6             | SELL | 5 @ 11.00  |
| Client C – OrderId 4 ★ MOVED | BUY | 100 @ 10.90 | Client B – OrderId 7             | SELL | 5 @ 11.10  |
| Client B – OrderId 3         | BUY | 5 @ 10.80   | Client B – OrderId 9 ✕ CANCELLED | SELL | 10 @ 11.20 |
| Client A – OrderId 2         | BUY | 20 @ 10.80  |                                  |      |            |

 **Price Modification Rule:** Changing the price of an order is equivalent to canceling the original and entering a brand-new order at the new price. OrderId 4 is removed from the 10.70 level and inserted at the back of the 10.90 bid queue — it gets the lowest priority among all orders at that new price.

 New insertion (price change)

 Cancelled / removed



# RULE: AGGRESSIVE ORDER — PARTIAL MATCH

Step 5a of 5

Action: Client C sends SELL 50 @ 10.90 → aggressive order sweeps bids from highest price first

| BEFORE               |                 |
|----------------------|-----------------|
| Client A – OrderId 1 | BUY 20 @ 10.90  |
| Client A – OrderId 8 | BUY 10 @ 10.90  |
| Client C – OrderId 4 | BUY 100 @ 10.90 |
| Client B – OrderId 3 | BUY 5 @ 10.80   |
| Client A – OrderId 2 | BUY 20 @ 10.80  |



| AFTER (sell 50 hits 10.90 bids)        |                |
|----------------------------------------|----------------|
| BIDS (remaining)                       |                |
| Client C – OrderId 4 (partial 80 left) | BUY 80 @ 10.90 |
| Client A – OrderId 3                   | BUY 5 @ 10.80  |
| Client B – OrderId 2                   | BUY 20 @ 10.80 |

Execution sequence (50 qty sell @ 10.90):

① OrderId 1 (BUY 20) → fully filled, removed from book    ② OrderId 8 (BUY 10) → fully filled, removed from book    ③ OrderId 4 (BUY 100) → partially filled 20 qty, remains with 80 qty remaining

# RULE: AGGRESSIVE ORDER — RESTING REMAINDER

Step 5b of 5

Action: Client A sends BUY 10 @ 11.00 → matches asks at 11.00, leftover BUY qty rests as new passive bid

AFTER — new passive bid rests at 11.00

| BIDS (Buy Orders)              |     |    |         | ASKS (Sell Orders)   |      |   |         |
|--------------------------------|-----|----|---------|----------------------|------|---|---------|
| Client A – OrderId 9 ★ RESTING | BUY | 4  | @ 11.00 | Client B – OrderId 7 | SELL | 5 | @ 11.10 |
| Client C – OrderId 4           | BUY | 80 | @ 10.90 |                      |      |   |         |
| Client A – OrderId 3           | BUY | 5  | @ 10.80 |                      |      |   |         |
| Client B – OrderId 2           | BUY | 20 | @ 10.80 |                      |      |   |         |

Execution: OrderId 5 (SELL 1 @ 11.00) fully matched ① → OrderId 6 (SELL 5 @ 11.00) fully matched ② → remaining 4 qty on the aggressive BUY has no more asks at 11.00, so it rests as new passive bid OrderId 9 at 11.00

# ORDER BOOK RULES — SUMMARY

①

## New Order

Inserted at correct price level, behind all existing orders at that price (FIFO). Bids: highest→lowest. Asks: lowest→highest.

②

## Reduce Quantity

Order stays in its current queue position. Priority is preserved. Partial fills also reduce quantity with no priority loss.

③

## Increase Quantity

Order loses priority — equivalent to canceling and re-entering at the back of the queue at that price level.

④

## Price Change / Cancel

Changing price = cancel + new order at new price. Order is removed from old level and inserted at back of new level.

⑤

## Aggressive Order — Partial

Sweeps passive orders from most to least aggressive price. Partially matched passive orders remain with reduced quantity.

⑥

## Aggressive Order — Resting

If aggressor quantity exceeds available passive liquidity, the remaining unmatched quantity rests as a new passive order.

# MATCHING ENGINE — Components

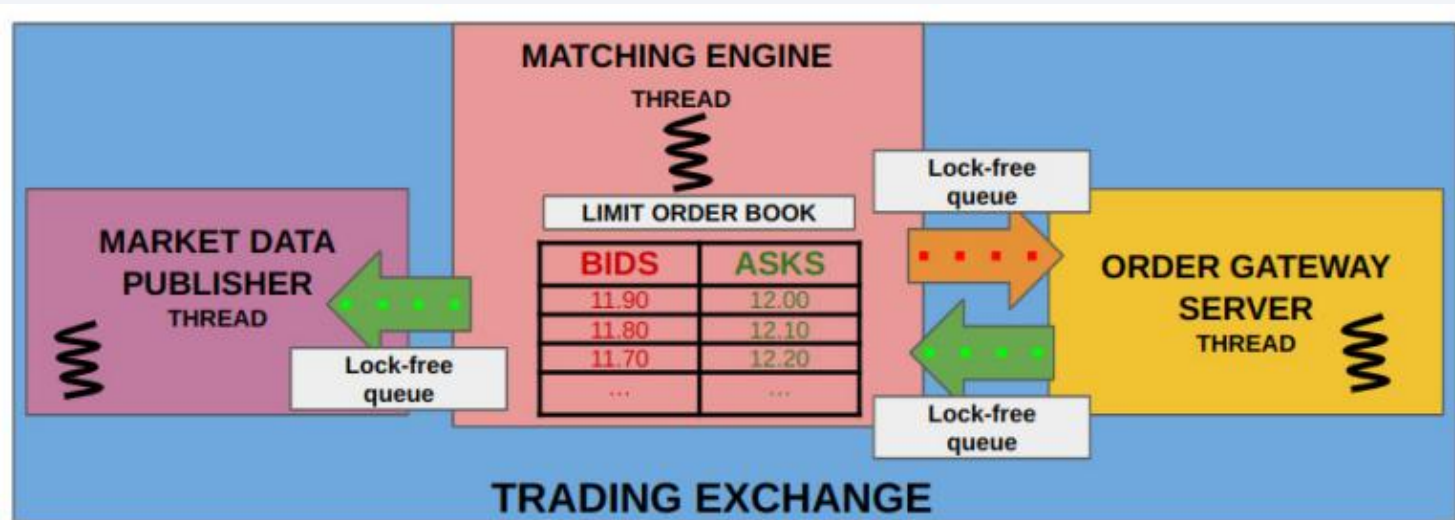


Figure 5.2 – Design of our matching engine component

# MATCHING ENGINE — DESIGN

## Threading Model

- ME, Market Data Publisher, Order Gateway = independent threads
- Enables max throughput during market bursts
- Each component operates without blocking others

## Communication

- Bi-directional lock-free queues between ME & Order Gateway
- One queue: ME → Market Data Publisher
- Lock-free FIFO queue (built in Ch. 4)

## Limit Order Book

- Sorted data structures for bids & asks
- Efficient insert / modify / delete on price levels
- Memory pool — no dynamic allocation on critical path

## Matching Logic

- Aggressive orders match passive from most to least aggressive price
- Partial fills: remaining passive qty stays in book
- Remaining aggressive qty rests as new passive order

# MARKET DATA PUBLISHER

Broadcasting the order book to all participants



# MARKET DATA PROTOCOL — MESSAGE FORMAT

## FIX TAG=VALUE EXAMPLE (new BUY order: 1,000 AAPL @ 175.16)

| TAG | FIX NAME       | VALUE        | DESCRIPTION                       |
|-----|----------------|--------------|-----------------------------------|
| 268 | NoMDEntries    | 1            | Number of updates in this message |
| 279 | MDUpdateAction | 0 (New)      | Add / Modify / Delete             |
| 269 | MDEntryType    | 0 (Bid)      | Bid or Ask side                   |
| 48  | SecurityID     | 68475 (AAPL) | Integer ID for the instrument     |
| 270 | MDEntryPx      | 175.16       | Price of this update              |
| 271 | MDEntrySize    | 1000         | Quantity of this update           |

*FIX uses human-readable TAG=VALUE pairs — flexible but slower to parse than binary formats.*

## PROTOCOL COMPARISON

### FIX / FAST



TAG=VALUE text. Most common in practice. Complex to decode — higher latency.

### ITCH / PITCH



Binary message-based. Common in US equities. Simpler than FAST.

### EOBI / SBE



Packed binary structs. Decode = reinterpret byte stream. Fastest.

# MARKET UPDATE MESSAGE TYPES — DETAIL



## Market State Changes

- Notifies participants of exchange state transitions
- States: Closed, Pre-Open, Opening/Auction, Trading
- Affects which order operations are permitted at each state



## Instrument Updates

- Min price increment (tick size): e.g. 0.10 — prices must be multiples
- Tick size value: profit per tick (1× for stocks; can differ for futures/options)
- $P\&L = ((\text{sell} - \text{buy}) / \text{min-increment}) \times \text{qty} \times \text{tick-size}$



## Order Updates

- Order Add: new passive order — instr-id, order-id, price, side, qty, FIFO priority
- Order Modify: qty or price changed; new priority except on qty reduction
- Order Delete: passive order removed — instr-id + order-id to identify it



## Trade Messages

- Signals an aggressive-passive match at the exchange
- Attributes: instr-id, aggressor side, execution price, traded qty
- Accompanied by Order Delete/Modify msgs to reflect resulting book changes



## Market Statistics

- Optional messages — not published by all exchanges
- Includes: total traded volume, open interest, OHLC prices for instrument
- Not on critical latency path — useful for analytics and reporting only

*In our implementation: Order Add / Modify / Delete + Trade messages — sufficient for a complete functional exchange simulation.*



# MARKET DATA PUBLISHER — ARCHITECTURE OVERVIEW



## Market Data Protocol Encoder

- Consumes order book changes from the Matching Engine via lock-free queue
- Encodes internal ME format → public market data binary protocol
- Publishes encoded updates to the Incremental UDP multicast stream
- Also forwards encoded updates to the Snapshot Synthesizer



## Snapshot Synthesizer

- Consumes encoded updates from the Protocol Encoder
- Continuously rebuilds a complete image of the current order book
- Periodically publishes full book snapshots to the Snapshot UDP stream
- Stamps each snapshot with the last incremental sequence number used

*Key design: Snapshot Synthesizer runs in its own thread — snapshot publishing never blocks or delays the incremental stream.*

# ORDER GATEWAY SERVER — DESIGN

TCP — reliable, in-order delivery of private order updates between exchange and each market participant

## TCP Connection Manager

- Accepts incoming TCP connections from participants
- Detects disconnections, maintains active connection list
- Routes order responses back to the correct client

## FIFO Sequencer

- Ensures fairness: processes requests in arrival order
- Serializes across multiple simultaneous client connections
- First In, First Out — prevents front-running

## Protocol Decoder & Encoder

- Translates exchange binary protocol ↔ internal ME format
- Packs/unpacks binary structures for order requests
- Minimal overhead — no complex parsing required

*Key difference vs. Market Data: order gateway is private & bidirectional. Only impacted participants are notified.*

# MARKET PARTICIPANT SIDE

Consuming data and sending orders



# MARKET DATA CONSUMER — DESIGN

## SYNCHRONIZATION PROCESS (Startup & Packet Drop Recovery)

1

### Clear Book

Reset local order book  
to empty state

2

### Subscribe Snapshot

Join snapshot multicast  
stream

3

### Queue Incremental

Buffer all incoming  
incremental updates

4

### Receive Full Snapshot

Wait for complete book  
state from snapshot  
stream

5

### Apply Queued Updates

Replay buffered  
incremental msgs on  
top of snapshot

## SUBCOMPONENTS

### Snapshot + Incremental Synchronizer

Manages stream subscriptions, detects packet sequence gaps,  
drives the recovery/sync procedure above

### Market Data Protocol Decoder

Translates exchange binary format (e.g. SBE/EOBI) to internal  
normalized format for the trading engine

# ORDER GATEWAY CLIENT — DESIGN

## TCP Connection Manager

- Connects to exchange order gateway server
- Maintains single reliable TCP connection
- Heartbeat mechanism for liveness detection
- Sequence numbers for application-level reliability

## Protocol Encoder & Decoder

- Encodes internal order requests → exchange format
- Decodes exchange responses → internal format
- Binary-packed structures for minimal overhead
- Translates execution notifications to strategy layer

Flow: Trading Strategy → Order Manager → [Lock-free queue] → OG Client → TCP → Exchange

Response path is reversed: Exchange → TCP → OG Client → [Lock-free queue] → Trading Engine

# ORDER GATEWAY CLIENT — PROTOCOL CODEC & MESSAGE FLOW



## FULL ORDER ROUND-TRIP FLOW

|        |                                              |
|--------|----------------------------------------------|
| CLIENT | ↓ Trading Strategy decides to send order     |
| CLIENT | ↓ Order Manager validates + tracks state     |
| CLIENT | ↓ Encoder packs into binary exchange format  |
| WIRE   | ↓ TCP transmission → exchange                |
| EXCH   | ↓ FIFO Sequencer + Matching Engine processes |
| WIRE   | ↓ TCP response ← exchange                    |
| CLIENT | ↓ Decoder unpacks exchange response          |
| CLIENT | ↓ Order Manager updates state + position     |
| CLIENT | ↓ Trading Strategy reacts to fill / reject   |

# TRADING STRATEGY FRAMEWORK

Where intelligence and execution meet



# TRADING STRATEGY FRAMEWORK — COMPONENTS

## Limit Order Book

Builds & maintains participant-side view of the book. Supports BBO/TOB synthesis, FIFO queue position tracking, and execution detection from public feed.

## Feature Engine

Computes trading signals and statistical predictive features from the order book. Updated on price, liquidity changes, and trade events.

## Trading Strategy

Makes trading decisions based on features, order book state, FIFO position, and risk. Supports market-making (passive) and liquidity-taking (aggressive) algorithms.

## Order Manager

Tracks live order states, handles in-flight conditions (e.g. cancel vs simultaneous fill), manages positions and partial/full executions.

## Risk Manager

Enforces: max order qty, max position, worst-case position, max realized/unrealized loss, traded volume caps, and rate-based order/loss limits.



# RISK MANAGEMENT — KEY METRICS

## Max Order Quantity

Prevents Fat Finger errors — a cap on the size of any single order

## Max Position

Limits the realized position held in a trading instrument

## Worst-Case Position

Position + all pending buy/sell orders — accounts for live orders

## Max Realized & Unrealized Loss

Tracks P&L in real-time, including open position mark-to-market

## Max Traded Volume

Prevents runaway algorithms from overtrading due to bugs

## Rate-Based Limits

Sliding window checks: orders/sec, trades/sec, loss/period

# SUMMARY

---

- 1 Exchange: Matching Engine + Market Data Publisher + Order Gateway Server
- 2 Participant: Market Data Consumer + Order Gateway Client + Trading Engine
- 3 Lock-free queues & memory pools used throughout for zero-allocation paths
- 4 UDP multicast for market data; TCP for private order gateway communications
- 5 Snapshot + incremental streams solve the order book synchronization problem
- 6 Trading framework: order book → feature engine → strategy → order/risk manager

# Appendix sample protocols

## FIX Protocol

### WIRE FORMAT

8=FIX.4.2|35=D|49=CLI|54=B|44=123.45|38=10

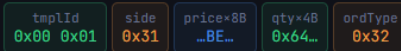
### MEMORY PATH FOR PRICE



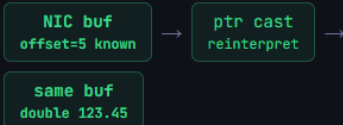
- 1 Scan for 44= tag — O(n) byte scan  
→ no fixed offset, position unknown
- 2 Extract "123.45" until SOH  
→ heap allocation: new String object
- 3 Call `parseDouble("123.45")`  
→ character-by-character ASCII math
- 4 Discard String, GC eventually collects  
→ GC pressure on every message

## SBE (Simple Binary Encoding)

### WIRE FORMAT — FIXED-LAYOUT BINARY



### MEMORY PATH FOR PRICE



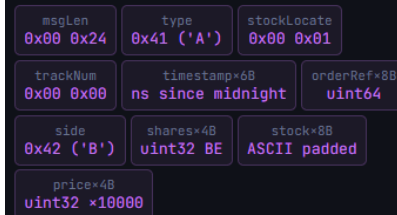
- 1 Offset computed from schema at compile time  
→ constant: `priceOffset = 5`
- 2 Pointer cast: `*(double*)(buf+5)`  
→ zero allocation, zero parse

**0 copies. O(1) field access. No allocations.**

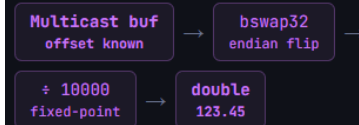
Transport: TCP (FIXP) or UDP. Bidirectional — designed for order entry and execution reports. Schema-driven, versioned.

## ITCH

### WIRE FORMAT — BIG-ENDIAN BINARY (ADD ORDER)



### MEMORY PATH FOR PRICE



- 1 Offset known from spec (fixed-layout like SBE)  
→ no scan needed
- 2 Read uint32 at offset, byte-swap BE→LE  
→ single CPU instruction: `bswap`
- 3 Divide by 10,000 (fixed-point price scale)  
→ no floating-point on the wire at all